Mini-project 1 in Advanced Machine Learning (02460)

by Frederik Svane (s224766), Marcus Christoffersen (s224750),
Nicholas Larsen (s224175) & Thor Højhus (s224233) on 05 March 2026

Part A: VAE Priors

We train VAEs (Kingma and Welling, 2014) with a Bernoulli likelihood on binarised MNIST, using three priors: a standard Gaussian, a 10-component MoG, and a flow-based prior (8 affine coupling layers (Dinh et al., 2017)). Encoder and decoder are symmetric 3-layer MLPs (784–512–512, $M = 32$, ReLU); for non-Gaussian priors the KL is estimated by Monte Carlo.

Figure 1 shows t-SNE (van der Maaten and Hinton, 2008) projections of the aggregate posterior and prior. The Gaussian prior sits as single probability mass over the posterior, where the MoG and Flow priors reshape themselves to track the digit clusters. The test ELBOs however barely separate (Gaussian -88.8 ± 0.7, MoG -88.2 ± 0.7, Flow -88.1 ± 0.4; mean±std over 5 runs), indicating the shared decoder is the bottleneck: when reconstruction saturates, a more expressive prior has little to improve. The flow prior trains the most stable (lowest variance).

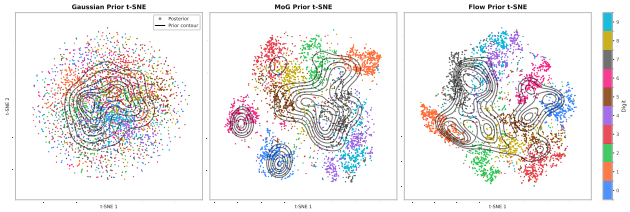

Figure 1: t-SNE projections of aggregate posterior (coloured by digit class) and prior density contours for Gaussian (left), MoG (middle), and Flow (right) priors.

Part B: Sampling Quality

We compare three generative pipelines on continuous MNIST. The **DDPM** (Ho et al., 2020) pairs a five-level U-Net (Ronneberger et al., 2015) (32, 64, 128, 256, 256 channels, skip connections, LogSigmoid, $T = 100$) with a linear noise schedule. The **latent DDPM** (Rombach et al., 2022) first trains a β-VAE (same MLP backbone as Part A but with a Gaussian likelihood, $M = 32$), then fits a fully-connected DDPM ($T = 1000$) to the encoder means. At test time it draws latent samples from the diffusion model and pushes them through the frozen decoder.

Pixel-space DDPM gives the best FID (Heusel et al., 2017) (36.96) but only manages 35 samples/s (Table 1). The Gaussian VAE is 370× faster and not far behind in FID (42.30). Latent DDPM runs diffusion in 32 dimensions instead of 784, increasing throughput to ∼8k samples/s, though the best FID (81.91 at $\beta = 10^{-3}$) lags both baselines. The choice of β matters: too small (10^{-6}) and the posterior is diffuse, giving the latent DDPM a hard target (FID 115), too large (≥ 0.1) and reconstruction quality drops (FID > 88).

Figure 2: 2×2 samples: VAE Gaussian (left), DDPM U-Net (mid), Latent DDPM $\beta = 10^{-3}$ (right).

Table 1: FID (↓) and sampling speed on standard MNIST ($N = 1000$).

Model	FID	Samp./s
VAE (Gaussian)	42.30	13,179
DDPM (U-Net)	36.96	35
Lat. DDPM $\beta = 10^{-6}$	115.15	8,405
Lat. DDPM $\beta = 10^{-3}$	81.91	8,613
Lat. DDPM $\beta = 0.1$	88.80	8,584
Lat. DDPM $\beta = 1$	153.18	8,861

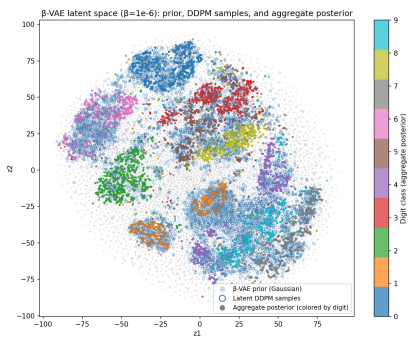

Figure 3: Latent space ($\beta = 10^{-6}$): Gaussian prior (grey), DDPM latents (blue), aggregate posterior (coloured).

Fig. 3 shows the latent space at $\beta = 10^{-6}$: the aggregate posterior (coloured) splits into ten digit clusters that the Gaussian prior (grey) does not match. The DDPM distribution (blue) covers the posterior mass much better, but overall FID remains high (115 vs. 42 for the VAE) because at this β the decoder itself is the limiting factor.

Code snippets

1. VAE ELBO with analytic / MC KL fallback

The `VAE.elbo` method uses `torch.distributions.kl_divergence` when closed-form KL is available (Gaussian prior) and falls back to a single-sample Monte Carlo estimate KL $\approx \log q_\phi(z|x) - \log p(z)$ for the MoG and flow priors.

```
def elbo(self, x):
    q = self.encoder(x)
    z = q.rsample()
    prior = self.prior()
    # Use analytic KL when available (Gaussian prior), fall back to MC for MoG
    try:
        kl = td.kl_divergence(q, prior)
    except NotImplementedError:
        kl = q.log_prob(z) - prior.log_prob(z)
    return torch.mean(self.decoder(z).log_prob(x) - self.beta * kl, dim=0)
```

Listing 1: VAE ELBO (vae_bernoulli.py)

2. Affine coupling layer (RealNVP-style flow)

Each `MaskedCouplingLayer` splits dimensions with a binary mask, applies scale and translation networks to the masked half, and accumulates the log-determinant for density evaluation.

```
def forward(self, z):
    x_masked = self.mask * z
    s = self.scale_net(x_masked)
    t = self.translation_net(x_masked)
    x = self.mask * z + (1 - self.mask) * (z * torch.exp(s) + t)
    log_det_J = ((1 - self.mask) * s).sum(dim=1)
    return x, log_det_J

def inverse(self, x):
    z_masked = self.mask * x
    s = self.scale_net(z_masked)
    t = self.translation_net(z_masked)
    z = self.mask * x + (1 - self.mask) * ((x - t) * torch.exp(-s))
    log_det_J = -((1 - self.mask) * s).sum(dim=1)
    return z, log_det_J
```

Listing 2: MaskedCouplingLayer forward pass (flow.py)

3. DDPM negative ELBO

At each training step a random timestep t is drawn, Gaussian noise ϵ is added to x_0 via the closed-form marginal, and the network predicts the noise. The loss is the mean squared error summed over data dimensions.

```
def negative_elbo(self, x):
    t = torch.randint(0, self.T, (x.shape[0],), device=x.device)
    epsilon = torch.randn_like(x)
    alpha_cumprod_t = self.alpha_cumprod[t].unsqueeze(-1)
    x_t = torch.sqrt(alpha_cumprod_t) * x + torch.sqrt(1 - alpha_cumprod_t) * epsilon
    t_normalized = t.unsqueeze(-1).float() / self.T
    epsilon_hat = self.network(x_t, t_normalized)
    neg_elbo = torch.sum((epsilon - epsilon_hat) ** 2, dim=-1)
    return neg_elbo
```

Listing 3: DDPM.negative_elbo (ddpm.py)

4. DDPM reverse sampling

Starting from $x_T \sim \mathcal{N}(0, I)$, the model iterates the DDPM reverse step, adding noise for $t > 0$ and returning the clean prediction x_0 at $t = 0$.

```
def sample(self, shape):
    x_t = torch.randn(shape).to(self.alpha.device)
    for t in range(self.T - 1, -1, -1):
        t_tensor = torch.full((shape[0], 1), t / self.T,
                              device=x_t.device)
        epsilon_hat = self.network(x_t, t_tensor)
```

```
        alpha_t = self.alpha[t]
        beta_t = self.beta[t]
        alpha_cumprod_t = self.alpha_cumprod[t]
        mu = (1 / torch.sqrt(alpha_t)) * (x_t - (beta_t / torch.sqrt(1 - alpha_cumprod_t)) *
    epsilon_hat)
        if t > 0:
            x_t = mu + torch.sqrt(beta_t) * torch.randn_like(x_t)
        else:
            x_t = mu
    return x_t
```

Listing 4: DDPM.sample (ddpm.py)

5. Latent DDPM: encoding and sampling

The β-VAE encoder maps training images to latent means; a fully-connected DDPM is trained on these vectors. At test time, the DDPM samples a latent which the frozen decoder maps to pixel space.

```
def encode_dataset(vae, data_loader, device):
    vae.eval()
    latents = []
    with torch.no_grad():
        for x, _ in data_loader:
            x = x.to(device)
            q = vae.encoder(x)
            z = q.mean  # use mean, not sample, for stable training
            latents.append(z.cpu())
    return torch.cat(latents, dim=0)

# --- training ---
latents     = encode_dataset(vae, train_loader, device)
latent_loader = DataLoader(latents, batch_size=B, shuffle=True)
ddpm = DDPM(FcNetwork(M, hidden), T=T).to(device)
train_ddpm(ddpm, optimizer, latent_loader, epochs, device)

# --- sampling ---
with torch.no_grad():
    z       = ddpm.sample((n, M))
    samples = vae.decoder(z).mean.clamp(0, 1)
```

Listing 5: Latent DDPM encode + sample (latent_ddpm.py)

Declaration of use of generative AI

This declaration **must** be filled out and included as the 3rd page of the document. The questions apply to all parts of the work, including research, project writing, and coding.

- I/we have used generative AI tools: yes

If you answered *yes*, please complete the following sections. List the generative AI tools you have used:

- Claude 4.6 Opus (via the web interface and the Claude Code CLI)

Describe how the tools were used:

What did you use the tool(s) for? Understanding theory behind VAEs, diffusion models, and flow-based priors; checking that our code implementations were correct; and assessing the quality and completeness of our written answers.

At what stage(s) of the process did you use the tool(s)? Throughout the project: during initial reading and theory review, during implementation and debugging, and during final report writing and review.

How did you use or incorporate the generated output? We used Claude as a discussion partner and reviewer. All code was written by us (except for the given templates) and verified independently. Claude helped us catch errors and clarify our understanding but did not generate code or text that was copied verbatim into the report.

References

L. Dinh, J. Sohl-Dickstein, and S. Bengio. Density estimation using Real NVP. In *International Conference on Learning Representations*, 2017.

M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

D. P. Kingma and M. Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014.

R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022.

O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention*, pages 234–241, 2015.

L. van der Maaten and G. Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9: 2579–2605, 2008.